

App Development Project Report

Title: Placement Portal

1. Student Details

Field	Details
Name	Ujjwal Singh
Roll Number	24f2007917
Email	24f2007917@ds.study.iitm.ac.in
About Me	I am a student of the IIT Madras BS Degree Program with a keen interest in full-stack web development, database systems, and building real-world applications that solve practical problems.

2. Project Details

Project Title: Placement Portal

Problem Statement

To design and build a comprehensive web-based Placement Portal that facilitates seamless interaction between students, companies, and administrators. The system enables students to register, upload resumes, browse placement drives, and track application status — while companies can post drives and manage applicants, and admins can oversee and approve all activities on the platform.

Approach

The application was built using Flask as the backend framework with a single-file modular structure. It implements a role-based access control system with three user types: Admin, Company, and Student. Raw SQLite queries are used for all database operations, following the project guidelines that restrict use of ORMs. The frontend uses Bootstrap 5 with Jinja2 templates for a responsive, clean interface.

3. AI/LLM Declaration

No AI/LLM tools were used in the development of this project.

All implementation logic, debugging, and integration were performed manually.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
SQLite (sqlite3 module)	Lightweight local database for storing all portal data
Jinja2	Template engine for rendering dynamic HTML pages
Bootstrap 5	Frontend styling and responsive design
Werkzeug	Password hashing and secure file handling
Flask Session	User authentication and session management
Python functools	Custom role-based access control decorators

5. Database Schema / ER Diagram

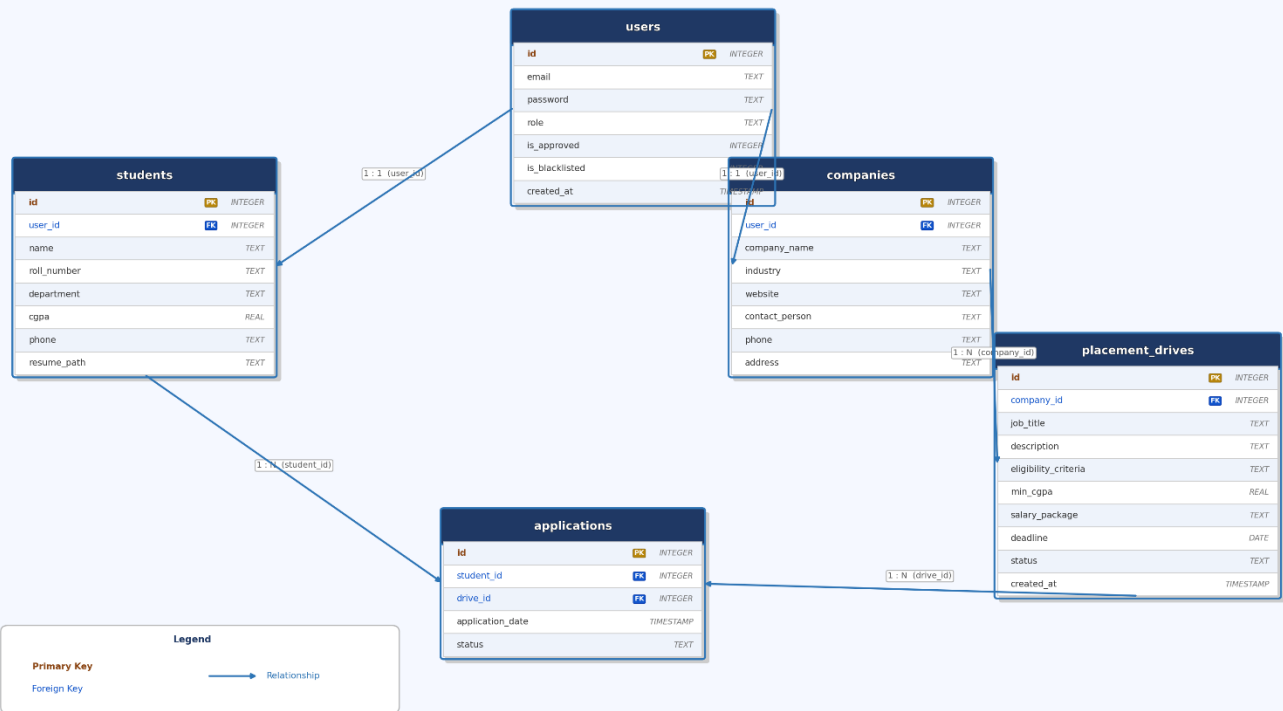
Tables

- 1. users** — Stores authentication data for all roles
 - id, email, password (hashed), role (admin/company/student)
 - is_approved, is_blacklisted, created_at
- 2. students** — Stores student profile information
 - id, user_id (FK), name, roll_number, department, cgpa, phone, resume_path
- 3. companies** — Stores company profile information
 - id, user_id (FK), company_name, industry, website, contact_person, phone, address
- 4. placement_drives** — Job postings/drives created by companies
 - id, company_id (FK), job_title, description, eligibility_criteria
 - min_cgpa, salary_package, deadline, status (pending/approved/rejected/closed), created_at
- 5. applications** — Student job applications
 - id, student_id (FK), drive_id (FK), application_date, status (applied/shortlisted/selected/rejected)
 - UNIQUE constraint: (student_id, drive_id) — prevents duplicate applications

Relationships

- One-to-One: users -> students (each student has one user account)
- One-to-One: users -> companies (each company has one user account)
- One-to-Many: companies -> placement_drives (a company can post many drives)
- One-to-Many: students -> applications (a student can apply to many drives)
- One-to-Many: placement_drives -> applications (a drive can have many applicants)

Placement Portal — Entity Relationship Diagram



6. API Resource Endpoints

Endpoint	Method	Description
/login	POST	Authenticate user and start session
/register/<role>	GET/POST	Register new student or company account
/logout	GET	Clear session and log out user
/admin/companies	GET	View and manage all registered companies
/admin/company/approve/<id>	GET	Approve a company registration
/admin/students	GET	View and manage all registered students
/admin/drives	GET	View all placement drives

/admin/drive/approve/<id>	GET	Approve a placement drive
/admin/applications	GET	View complete application history
/company/drive/create	GET/POST	Create a new placement drive
/company/drive/edit/<id>	GET/POST	Edit an existing placement drive
/company/drive/applicants/<id>	GET	View applicants for a specific drive
/company/application/update/<id>/<status>	GET	Update applicant status (shortlist/select/reject)
/student/drives	GET	Browse available approved placement drives
/student/apply/<drive_id>	POST	Submit application to a drive (with eligibility check)
/student/applications	GET	View personal application history and status
/student/profile	GET/POST	View and update student profile, upload resume
/uploads/<filename>	GET	Serve uploaded resume PDF files

7. Architecture and Features

Architecture Overview

Component	Description
app.py	Single-file Flask application — contains all routes, models, decorators, and DB init
templates/admin/	Jinja2 HTML templates for admin panel views
templates/company/	Jinja2 HTML templates for company-facing views
templates/student/	Jinja2 HTML templates for student-facing views
static/uploads/	Storage directory for student resume PDF uploads
placement_portal.db	SQLite database file, auto-created on first run

Implemented Features

Student Features:

- Registration and profile management with CGPA, department, and phone details
- Resume upload (PDF, up to 16MB) with secure file storage
- Browse approved placement drives with CGPA eligibility filtering
- One-click application to drives with duplicate application prevention
- Real-time application status tracking (Applied / Shortlisted / Selected / Rejected)
- Personal dashboard with application statistics

Company Features:

- Registration and company profile management
- Login after admin approval workflow
- Create placement drives with job title, description, eligibility criteria, salary, and deadline
- Edit and close active drives
- View all applicants per drive with student profiles and resume links
- Update per-applicant status (Shortlisted / Selected / Rejected)

Admin Features:

- Admin dashboard with real-time statistics on students, companies, drives, and applications
- Approve / reject / blacklist / delete company registrations
- Approve / reject / blacklist / delete student registrations
- Approve or reject placement drives submitted by companies
- Full visibility of all applications across the platform

Security & Technical Features:

- Role-based access control using custom Python decorators
- Password hashing using Werkzeug security utilities
- Session-based authentication with Flask session management
- Blacklist checking on every protected request
- Raw SQLite queries with parameterized inputs

8. Video Presentation

Drive Link: [Click here to view presentation video](https://drive.google.com/file/d/19nWW0Ikul9r7Yy49mZsIHYLsgWFerayB/view?usp=sharing) or copy and paste below link in browser
<https://drive.google.com/file/d/19nWW0Ikul9r7Yy49mZsIHYLsgWFerayB/view?usp=sharing>